ALLIANCE UNIVERSITY

GREEDY FORWARDING SCHEDULER FOR A RISC PIPELINE SIMULATOR

Submitted by
Dasari Saideep Srinivas
University Enrolment No. 2411021060596
Squad 70

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
Simran Kumari

October 26 2026



ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on Design and Analysis of Algorithms is submitted by Dasari Saideep Srinivas [Uni. Enrolment No. 2411021060596] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

Simran Kumari

Date: _____ Signature: _____

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled GREEDY FORWARDING SCHEDULER FOR A RISC PIPELINE SIMULATOR, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Simran Kumari.

I further confirm that the views, findings and conclusions expressed here come from my own effort. This project is an integrated work shared with Formal Language and Automata Theory and Computer Organization and Architecture. Each subject has its own report. The DAA report covers the greedy forwarding algorithm and its optimality analysis.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Dasari Saideep Srinivas
2411021060596

Name of the Student University Registration Number Signature

_____ _____ _____

Acknowledgement

I would like to thank everyone who helped me finish this integrated microproject. This report covers the Design and Analysis of Algorithms component: the greedy forwarding scheduler.

I thank Simran Kumari for the guidelines and for making us combine concepts from Modules 1–3 across subjects. The DAA lectures on greedy algorithms, the greedy choice property, and exchange arguments gave me the reasoning I needed to argue that "always forward when possible, stall only when necessary" is optimal for an in-order pipeline.

The activity selection problem from DAA Module 3 was the closest analogy. In activity selection, you always pick the earliest finishing activity. In pipeline forwarding, you always pick the shortest available forwarding path. Both are greedy, both are provably optimal for their setting, and both fail if the setting changes (out-of-order execution for forwarding, weighted intervals for activity selection).

Thank you.

Dasari Saideep Srinivas

TABLE OF CONTENTS

Acknowledgement 4
Abstract 6
Chapter I – Introduction 7
  Problem Statement 7
  Theory 8
Chapter II – System Configuration 9
  Project Links 9
  Greedy Algorithm 9
  Source Code 10
Chapter III – Results 11
Chapter IV – Conclusion 13
References 13

Abstract

This report covers the Design and Analysis of Algorithms component of an integrated microproject. The project is a RISC pipeline simulator. The DAA piece is the forwarding scheduler: the algorithm that decides, for each instruction in the decode stage, whether it can receive its operand via a forwarding path or must stall. The decision is greedy.

The greedy strategy is: for each source register, check the shortest forwarding path first (EX→EX), then the next shortest (MEM→EX). If a path provides the needed value, take it. Only stall when no forwarding path is available (load-use case). This is a locally optimal choice at each cycle, and it is also globally optimal for an in-order single-issue pipeline because no reordering of instructions is possible. The proof follows the exchange argument template from DAA Module 3.

The report explains the greedy choice property, the exchange argument, and shows experimental results: with greedy forwarding the simulator produces fewer stalls and lower CPI than without forwarding, on every test program. A forwarding log table records each greedy decision for traceability.

Chapter I – Introduction

In DAA Module 3, we learn that a greedy algorithm builds a solution step by step, choosing at each step the option that looks best locally. The greedy choice property says: a locally optimal choice at each step leads to a globally optimal solution. Activity selection, Huffman coding, and Kruskal's MST all work this way.

Pipeline forwarding is a scheduling problem. At each cycle, the scheduler faces a decision: does the instruction in ID need a value that is not yet in the register file? If yes, can it be forwarded from a later pipeline stage? The greedy approach says: always forward if the path exists, never stall unnecessarily. The alternative is to stall for every RAW hazard (no forwarding) or to reorder instructions (out-of-order execution, which is not greedy but dynamic programming / search-based).

I chose this angle because it shows a real greedy algorithm inside a system that is usually taught as a hardware concept. The forwarding logic is not hardware here; it is a software scheduler that makes the same decision a hardware forwarding unit would make. That makes it a DAA problem in a COA setting.

Problem Statement

Given the current state of a five-stage pipeline (which instructions are in IF, ID, EX, MEM, WB) and their source and destination registers, decide at each cycle whether to forward a value or insert a stall. The goal is to minimise the total number of stalls (and therefore minimise CPI) using a greedy strategy: always pick the shortest available forwarding path.

Theory

Greedy choice property for forwarding: If the value needed by the instruction in ID is available from a forwarding path (EX→EX or MEM→EX), taking the forwarding path is at least as good as stalling. No alternative choice at this cycle can reduce total stalls below what greedy achieves.

Exchange argument: Suppose an optimal solution S* stalls at cycle k where greedy would have forwarded. Replace that stall with a forward. The instruction in ID advances one cycle earlier. No later instruction is affected negatively, because forwarding does not consume the value (it is still written to the register file at WB). So S* with the stall replaced is at least as good as S*. Therefore greedy is optimal.

This argument only holds for in-order, single-issue pipelines. If the processor could reorder instructions (out-of-order execution), the problem becomes a scheduling problem with dependencies, which is closer to a DAG scheduling problem and may need dynamic programming or list scheduling — not greedy. That is the boundary of the greedy approach, and it matches the DAA lectures where we identify when greedy fails.

Complexity: The hazard check at each cycle iterates over the source registers of the ID instruction (at most 2) and compares them with the destination register of the EX and MEM instructions (constant-time lookups). So the per-cycle cost is O(1), and the total cost over n instructions and n + k cycles is O(n + k) where k is the number of stalls. Since $k \leq n$ (at most one stall per instruction), the total is O(n).

Chapter II – System Configuration

Same platform as the FLAT and COA reports. The greedy scheduler is implemented inside the pipeline engine in js/pipeline.js. It is the checkHazards function called once per cycle during the ID stage.

Project Links

GitHub repository: open to view

Live deployment: open to view

Greedy Algorithm

Pseudocode for the greedy forwarding decision at cycle c:

```
GREEDY-FORWARD(ID_insn, EX_insn, MEM_insn):
  for each source register r of ID_insn:
    if r == R0: skip          # R0 is hardwired to 0
    if EX_insn writes r:
      if EX_insn is a LOAD:
        return STALL          # value not ready until MEM
      else:
        record FORWARD(EX → ID, r)
        continue              # greedy: take shortest path
    if MEM_insn writes r:
      record FORWARD(MEM → ID, r)
  return NO_STALL
```

The greedy choice is on line "continue": when the EX path is available, take it and stop looking. The MEM path is only checked if EX did not provide the value. This is the "earliest available" heuristic, analogous to "earliest finishing time" in activity selection.

Connection to DAA Module 3 concepts:

- **Greedy choice property** (LU 3.1): each forward/stall decision is locally optimal.
- **Exchange argument** (LU 3.1): replacing a stall with a forward never worsens the schedule.
- **Activity selection analogy** (LU 3.3): earliest finish → shortest forwarding path.
- **Optimal substructure**: the schedule up to cycle c is optimal, and the decision at cycle c+1 depends only on the current pipeline state, not on past decisions.

Source Code

The forwarding decision is in *pipeline-sim/js/pipeline.js*, function checkHazards (reproduced in the COA report). The DAA contribution is the reasoning behind the function, not a separate file. The greedy property is visible in the code: the function returns as soon as it finds a stall (load-use) or records a forward (EX path), and only falls through to the MEM check if the EX path did not match.

Chapter III – Results

The key experiment is the comparison between greedy forwarding (on) and no forwarding (off). The table below shows the same five test programs run with both settings.

Test	Forwarding ON (greedy)			Forwarding OFF		
	Cycles	Stalls	CPI	Cycles	Stalls	CPI
Data hazard (3 insns + HLT)	8	0	2.00	12	4	3.00
Load-use (2 insns + HLT)	8	1	2.67	10	3	3.33
No hazards (4 insns + HLT)	9	0	1.80	9	0	1.80
Branch (4 insns + HLT)	10	0	2.00	12	2	2.40
Complex mix (8 insns + HLT)	14	1	1.56	20	7	2.22

Observations:

1. Greedy forwarding never produces more stalls than no-forwarding. This is expected: forwarding can only remove stalls, never add them.
2. The "No hazards" test is identical in both modes. There is nothing to forward, so the greedy decision is never triggered. This confirms that the algorithm has zero overhead when it is not needed.
3. The "Complex mix" test shows the largest improvement: from 7 stalls down to 1. The single remaining stall is a load-use hazard that no amount of forwarding can remove without reordering instructions.
4. CPI improves in every test where hazards exist. The complex mix drops from 2.22 to 1.56, which is a 30% improvement.

Forwarding log (Complex mix, first 4 events):

Cycle	Path	Register	Producer instruction
C4	EX→ID	R2	ADDI R2, R0, 20
C5	EX→ID	R3	ADD R3, R1, R2
C8	MEM→ID	R4	LW R4, 0(R0)
C9	EX→ID	R5	SUB R5, R4, R1

Each row is a greedy decision. At C4, the scheduler chose EX→ID for R2 because the ADDI producing R2 was in EX. At C8, it used MEM→ID for R4 because the load was in MEM (the data was just read from memory). No decision was wrong, and no decision could have been better. That is the greedy guarantee.

Chapter IV – Conclusion

The greedy forwarding scheduler does what it was supposed to do: at each cycle it picks the shortest available forwarding path, and only stalls when no path exists. The exchange argument shows this is optimal for an in-order pipeline. The experiments confirm the theory: greedy forwarding always matches or beats no-forwarding, and the single unavoidable stall (load-use) is the provable lower bound.

What I took from this part of the project is that "greedy" is not just a chapter heading. It is a way to think about any decision that is made once per step and cannot be undone. The pipeline scheduler makes one decision per cycle. It cannot go back and un-stall a previous cycle. The greedy choice property says that is fine, as long as each decision is locally optimal. The exchange argument is how I prove it. That reasoning pattern — state the property, show the exchange, identify where it breaks — is the same for activity selection, for Huffman, and for Dijkstra. The pipeline just gave me one more place to use it.

The limit is clear: greedy works for in-order, single-issue. If the pipeline were superscalar or out-of-order, the scheduler would need to consider multiple instructions per cycle and reorder them, which is a different class of problem (closer to list scheduling or Tomasulo's algorithm). That is where greedy stops and more complex algorithms begin.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. (Chapter 16: Greedy Algorithms.)
2. Kleinberg, J., & Tardos, É. (2005). *Algorithm design*. Addison-Wesley. (Chapter 4: Greedy Algorithms — exchange arguments.)
3. Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design: The hardware/software interface* (RISC-V ed.). Morgan Kaufmann. (Section 4.7: Data hazards and forwarding.)